

ES55 Project

Description:

The objective of this project is to numerically model the transient cooling behavior of a hot beverage contained in a cup and exposed to ambient air. This system was selected because it represents a realistic physical process that can be modeled using fundamental heat transfer principles while also lending itself naturally to the application of numerical methods.

The liquid is assumed to behave as a lumped thermal mass, meaning that its temperature is spatially uniform at any given time. This assumption is justified by the relatively high thermal conductivity of water and the small characteristic length scale of the system, which together result in a low Biot number. Under these conditions, the lumped-capacitance approximation provides a reasonable first-order model of the cooling process.

Cooling occurs primarily through convection between the cup and the surrounding air. The governing energy balance yields a first-order ordinary differential equation for temperature as a function of time. This equation is solved using multiple numerical techniques learned throughout the course and compared against an analytical solution for validation.

Beyond simply computing temperature decay, this project explores how physical parameters and numerical discretization affect system behavior. Parameter studies and visualization are used to reinforce physical intuition and numerical accuracy.

Inputs:

The model requires physical, thermal, and numerical inputs that define the system and control the simulation.

Physical Properties

- Liquid mass, m (kg)
- Specific heat capacity, c_p (J/kg*K)

Geometric Properties

- Effective convective surface area, A (m²)

Thermal Environment

- Initial temperature, T_0 (K)
- Ambient temperature, T_∞ (K)
- Convective heat transfer coefficient, h (W/m²*K)

Numerical Parameters

- Total simulation time (s)
- Time step size for explicit methods (s)

Outputs

The numerical model produces the following outputs:

- Liquid temperature as a function of time, $T(t)$
- Temperature and time comparison plots for different numerical methods
- Parameter study plots showing sensitivity to h and time step size
- Qualitative and quantitative comparison between numerical and analytical solutions

Equations and Assumptions

Energy Balance (Lumped Capacitance)

$$mc_p \frac{dT}{dt} = -hA(T - T_\infty)$$

Dividing by m_p :

$$\frac{dT}{dt} = -\frac{hA}{mc_p}(T - T_\infty)$$

Assumptions

- Temperature within the liquid is spatially uniform
- Thermophysical properties are constant
- Heat loss is dominated by convection
- Radiation and conduction are neglected

Numerical Methods

The governing first-order ordinary differential equation was solved using multiple numerical techniques.

- Euler Method

The Euler method approximates the time derivative using a forward finite difference:

$$T_{i+1} = T_i - \frac{hA}{mc_p}(T_i - T_\infty)\Delta t$$

This method is simple to implement and serves as a baseline numerical solution. However, it is conditionally stable and sensitive to the time-step size, as demonstrated by a numerical sensitivity study.

- Heun's Method

Heun's method is a second-order predictor-corrector scheme that improves upon Euler's method by averaging the slope at the beginning and end of each time step.

Predictor step:

$$T^* = T_i - k(T_i - T_\infty)\Delta t$$

Corrector step:

$$T_{i+1} = T_i - \frac{\Delta t}{2} (k(T_i - T_\infty) + k(T^* - T_\infty))$$

This method significantly reduces truncation error while remaining computationally efficient.

- Runge–Kutta Method (ode45)

MATLAB's ode45 solver, based on an adaptive Runge-Kutta method, was used to obtain a highly accurate numerical solution. Because of its adaptive time stepping and stability, the ode45 solution was treated as the primary numerical reference.

- Analytical Solution

The governing equation is linear, an analytical solution exists:

$$T(t) = T_\infty + (T_0 - T_\infty)e^{-\frac{hA}{mc_p}t}$$

This solution was used to validate all numerical methods.

Validation of Results

Validation was performed through analytical comparison and logical parameter variation. The ode45 and analytical solutions were nearly indistinguishable across the full simulation time, confirming correct implementation of the governing equation. Euler's method exhibited increasing deviation with larger time steps, whereas Heun's method demonstrated improved accuracy and convergence.

Logical validation was also performed by varying physical parameters. Increasing the convective heat transfer coefficient led to faster cooling, whereas increasing liquid mass slowed it. These trends are consistent with fundamental heat transfer theory.

Parameter Study Results

Case A – Low Convection (Natural Convection)

A low value of h produced slow, gradual cooling. The temperature decayed smoothly toward ambient conditions, demonstrating physically realistic behavior.

Case B – High Convection (Forced Convection)

Increasing h significantly increased the cooling rate, confirming that convective heat loss scales directly with the convection coefficient.

Case C – Time-Step Sensitivity

For Euler's method, large time steps produced numerical instability and non-physical temperature behavior. Reducing the time step improved accuracy and convergence, highlighting the limitations of first-order explicit methods.

Discussion

This project demonstrates how numerical methods can be applied to model a realistic thermal system. The comparison between Euler, Heun, and Runge-Kutta methods highlights the tradeoff between simplicity and accuracy. While Euler's method is intuitive, higher-order methods are necessary for reliable predictions.

The lumped-capacitance assumption provides a reasonable approximation for this system, though future improvements could include modeling conduction through the cup walls or radiative heat loss.

Conclusion

The transient cooling of a hot beverage was successfully modeled using numerical methods. Multiple techniques were implemented, validated, and compared against an analytical solution. Parameter studies and visualization reinforced both physical intuition and numerical theory.

This project demonstrates the effective use of numerical methods to solve engineering problems involving ordinary differential equations and transient behavior.

References

- Chapra, S. C. (2007). *Applied Numerical Methods with MATLAB for Engineers and Scientists*. McGraw-Hill.
- Incropera, F. P., DeWitt, D. P., Bergman, T. L., & Lavine, A. S. (2011). *Fundamentals of Heat and Mass Transfer*. Wiley.
- MathWorks. *MATLAB ODE Suite Documentation*.

Appendix A: Sample Output

Case A: Low Convection (Natural Convection)

Inputs:

- $h = 10 \text{ W/m}^2\text{K}$
- $m = 0.35 \text{ kg}$
- $c_p = 4180 \text{ J/kgK}$
- $A = 0.025 \text{ m}^2$
- $T_0 = 368 \text{ K}$
- $T_\infty = 293 \text{ K}$

Output:

- Temperature decreases slowly, asymptotically approaching the ambient temperature.

- The cooling curve exhibits smooth exponential decay.
- Numerical solutions from ode45 and the analytical solution overlap closely.

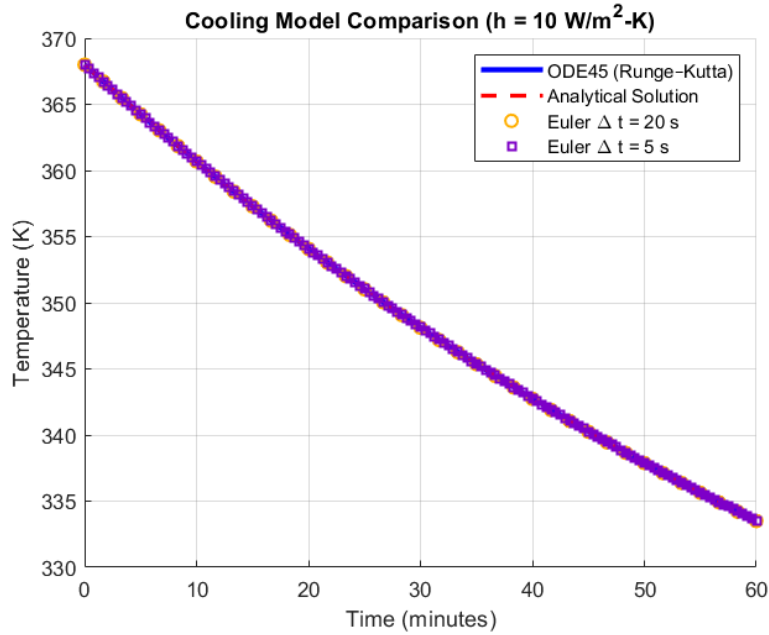


Figure 1: Cooling Model

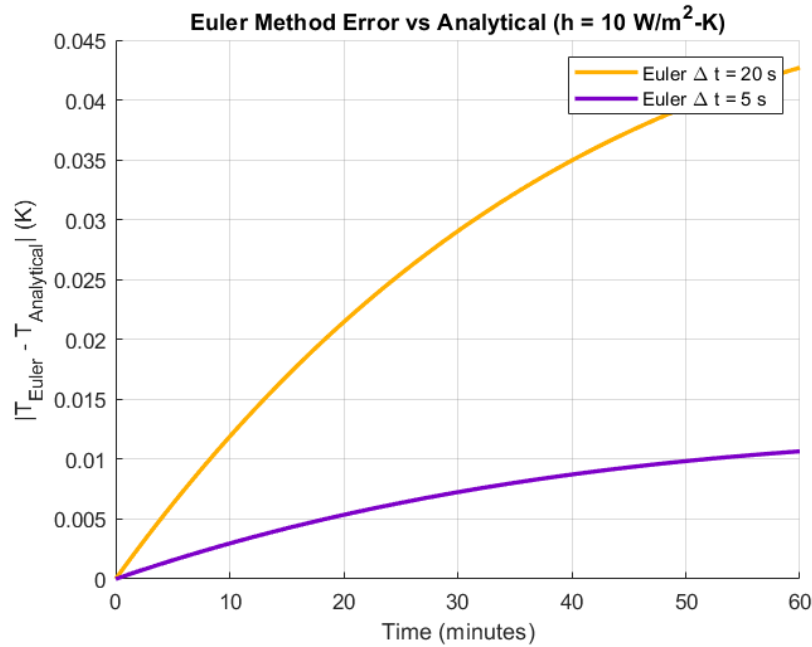


Figure 2: Euler Method Error vs Analytical

Physical Interpretation

This case represents weak natural convection. The low convective heat transfer coefficient limits heat loss, resulting in slow cooling.

Case B: High Convection (Forced Convection)

Inputs:

- Same as Case A except:
- $h = 50 \text{ W/m}^2\cdot\text{K}$

Output:

- Temperature decreases much more rapidly than in Case A.
- The beverage approaches ambient temperature in a significantly shorter time.
- Analytical and ode45 solutions remain in strong agreement.

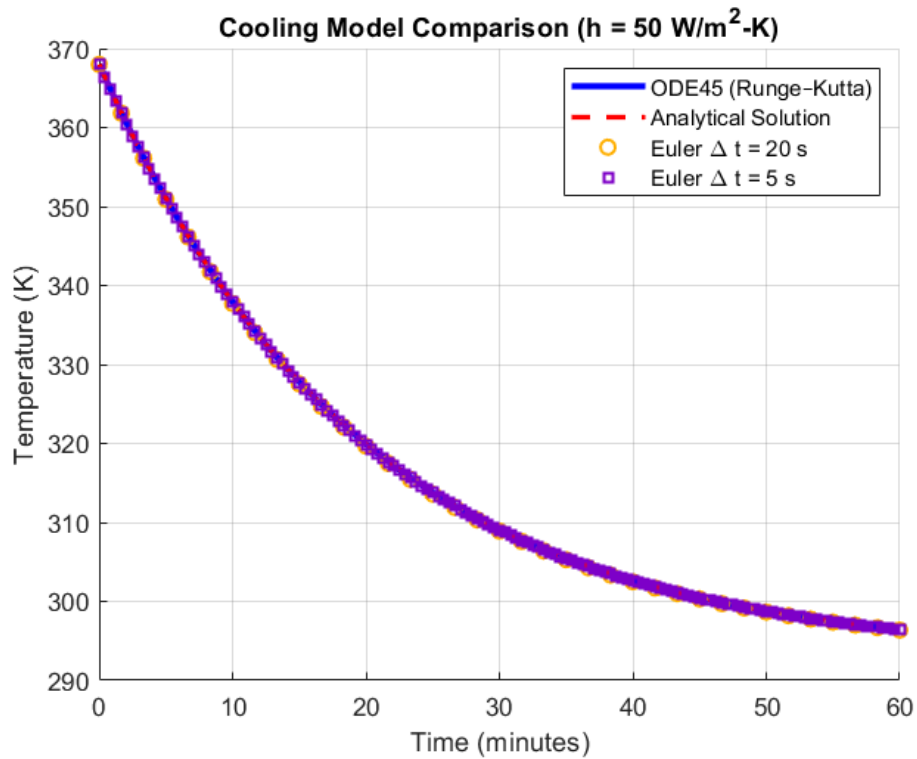


Figure 3: Cooling Model

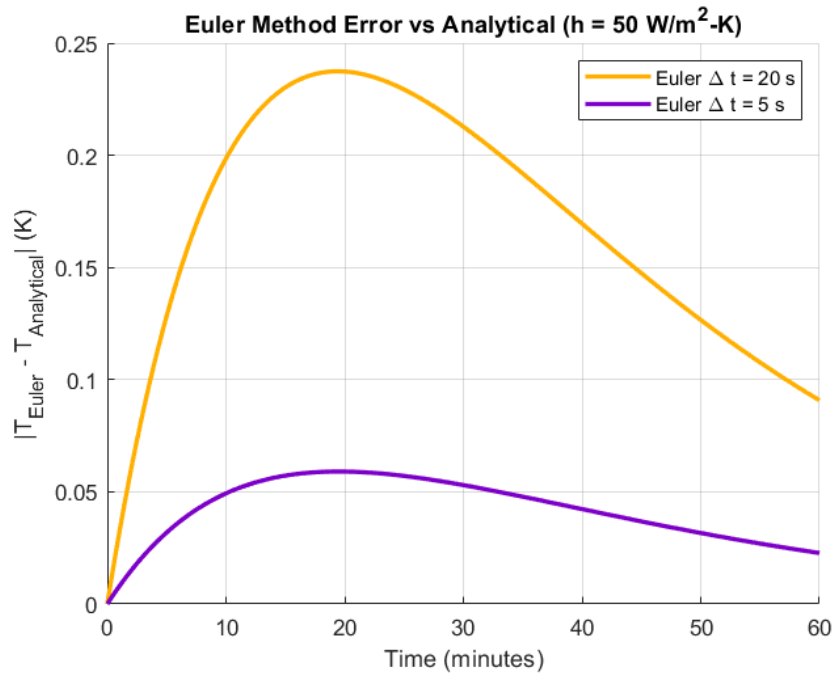


Figure 4: Euler Method Error vs Analytical

Physical Interpretation

Increasing the convection coefficient increases the rate of heat loss, consistent with heat transfer theory.

Case C: Euler Time-Step Sensitivity

Inputs:

- $h = 20 \text{ W/m}^2\cdot\text{K}$
- Euler time steps:
 - $\Delta t = 20 \text{ s}$
 - $\Delta t = 5 \text{ s}$

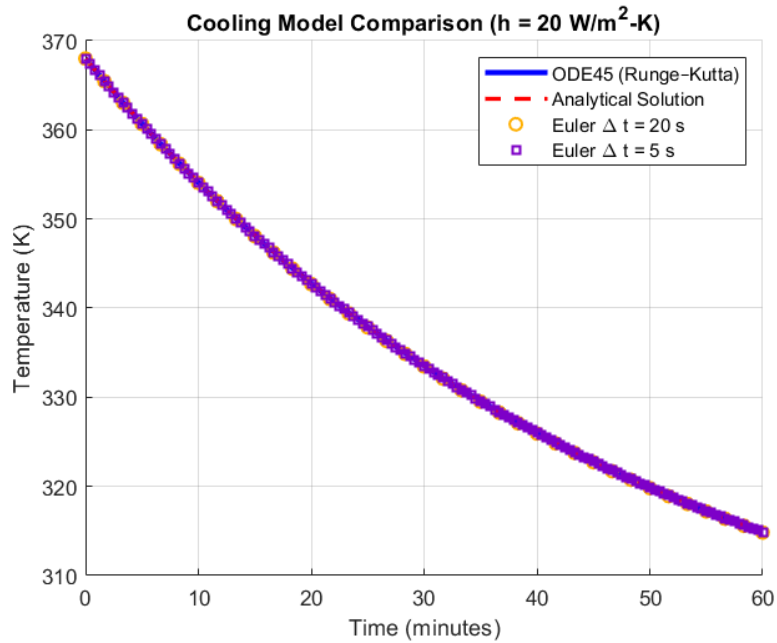


Figure 5: Cooling Model

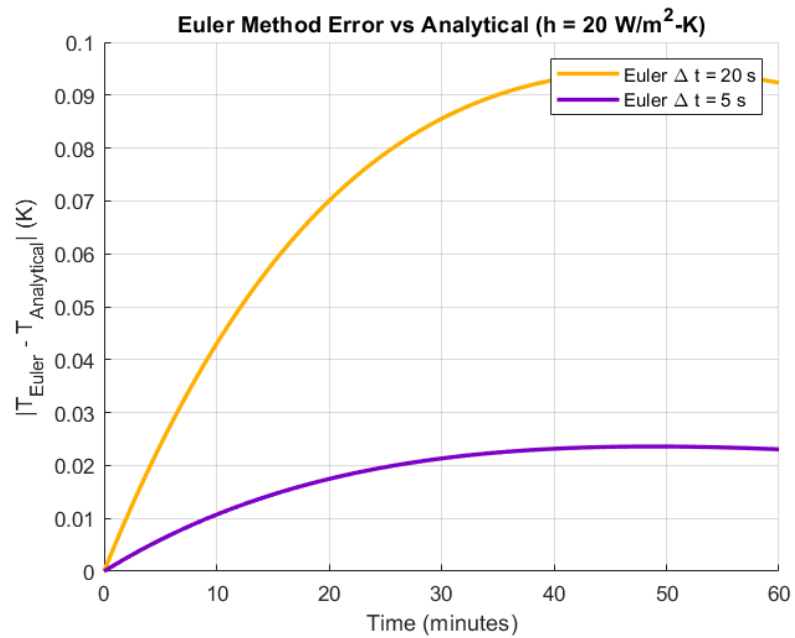


Figure 6: Euler Method Error vs Analytical

Output:

- Large time steps led to increased numerical error and reduced accuracy, whereas smaller time steps yielded smoother solutions that converged toward the analytical result.
- Smaller time-step sizes yield smoother curves that converge toward the analytical solution.

Numerical Interpretation

This case demonstrates the conditional stability of Euler's method and the importance of time-step selection.

Appendix B: MATLAB Code

```
%% Lemus_CoffeeCooling.m
```

```
clear; close all; clc;
```

```
m = 0.35; % mass (kg)
```

```
cp = 4180; % specific heat (J/kg-K)
```

```
A = 0.025; % effective area (m^2)
```

```
T0 = 368; % initial temperature (K)
```

```
Tinf = 293; % ambient temperature (K)
```

```
tEnd = 3600; % simulation time (s)
```

```
% C onvection coefficients
```

```
h_list = [10 20 50]; % W/m^2-K
```

```
for hh = 1:length(h_list) %goes through each convection
```

```
    h = h_list(hh);
```

```
    k = h*A/(m*cp); % ODE constant:  $dT/dt = -k(T - T_{inf})$ 
```

```
    odefun = @(t,T) -k*(T - Tinf);
```

```
    [t_ode, T_ode] = ode45(odefun, [0 tEnd], T0);
```

```
    T_ana = Tinf + (T0 - Tinf)*exp(-k*t_ode);
```

```
    %dt = 20s
```

```
    dt1 = 20;
```

```
    t_e1 = 0:dt1:tEnd;
```

```
    T_e1 = zeros(size(t_e1));
```

```
    T_e1(1) = T0;
```

```
    for i = 2:length(t_e1)
```

```
        T_e1(i) = T_e1(i-1) - k*(T_e1(i-1) - Tinf)*dt1;
```

```
    end
```

```
    %dt = 5s
```

```
    dt2 = 5;
```

```
    t_e2 = 0:dt2:tEnd;
```

```
    T_e2 = zeros(size(t_e2));
```

```
    T_e2(1) = T0;
```

```
    for i = 2:length(t_e2)
```

```
        T_e2(i) = T_e2(i-1) - k*(T_e2(i-1) - Tinf)*dt2;
```

```
    end
```

```
    figure; hold on; grid on;
```

```
    plot(t_ode/60, T_ode, 'b-', 'LineWidth', 2.5); % ODE45
```

```
    plot(t_ode/60, T_ana, 'r--', 'LineWidth', 2.0); % Analytical
```

```

% Euler
step_mark = 5; % show every 5th point
plot(t_e1(1:step_mark:end)/60, T_e1(1:step_mark:end), 'o', ...
     'Color', [1 0.7 0], 'MarkerSize', 6, 'LineWidth', 1.2);
plot(t_e2(1:step_mark:end)/60, T_e2(1:step_mark:end), 's', ...
     'Color', [0.5 0 0.8], 'MarkerSize', 5, 'LineWidth', 1.2);
xlabel('Time (minutes)');
ylabel('Temperature (K)');
title(sprintf('Cooling Model Comparison (h = %.0f W/m^2-K)', h));
legend( ...
     'ODE45 (Runge-Kutta)', ...
     'Analytical Solution', ...
     'Euler \Delta t = 20 s', ...
     'Euler \Delta t = 5 s', ...
     'Location','northeast');
%Euler vs Analytical
Tana_e1 = Tinf + (T0 - Tinf)*exp(-k*t_e1);
Tana_e2 = Tinf + (T0 - Tinf)*exp(-k*t_e2);
figure; hold on; grid on;
plot(t_e1/60, abs(T_e1 - Tana_e1), '-', ...
     'Color', [1 0.7 0], 'LineWidth', 2);
plot(t_e2/60, abs(T_e2 - Tana_e2), '-', ...
     'Color', [0.5 0 0.8], 'LineWidth', 2);
xlabel('Time (minutes)');
ylabel('|T_{Euler} - T_{Analytical}| (K)');
title(sprintf('Euler Method Error vs Analytical (h = %.0f W/m^2-K)', h));
legend('Euler \Delta t = 20 s', 'Euler \Delta t = 5 s', 'Location','northeast');
end
disp('Simulation complete. ');
disp('Generated improved comparison plots + clean Euler error plots for each h case. ');

```

Appendix C: Mathematical Details

Governing Energy Balance

Assuming the liquid temperature is spatially uniform (lumped capacitance):

$$mc_p \frac{dT}{dt} = -hA(T - T_\infty)$$

Dividing by m_p :

$$\frac{dT}{dt} = -\frac{hA}{mc_p}(T - T_\infty)$$

Define:

$$k = \frac{hA}{mc_p}$$

Then:

$$\frac{dT}{dt} = -k(T - T_\infty)$$

Analytical Solution

Separate variables:

$$\frac{dT}{T - T_\infty} = -k dt$$

Integrate:

$$\ln|T - T_\infty| = -kt + C$$

Apply initial condition $T(0) = T_0$:

$$T(t) = T_\infty + (T_0 - T_\infty)e^{-kt}$$

Explicit Euler Method

Approximate derivative:

$$\frac{T_{i+1} - T_i}{\Delta t} = -k(T_i - T_\infty)$$

Update equation:

$$T_{i+1} = T_i - k(T_i - T_\infty)\Delta t$$

Heun's Method (Improved Euler)

Predictor step:

$$T^* = T_i - k(T_i - T_\infty)\Delta t$$

Corrector step:

$$T_{i+1} = T_i - \frac{\Delta t}{2} (k(T_i - T_\infty) + k(T^* - T_\infty))$$

Heun's method is second-order accurate and reduces truncation error relative to Euler's method.

Runge-Kutta

MATLAB's ode45 implements an adaptive Runge-Kutta method that dynamically adjusts the time step to maintain accuracy and stability. This method was used as the reference numerical solution.